

Code Review AI — Final Consolidated Report

Marlen Melis · Student ID 2026140428 · School of Computing Course: *Practical Techniques in Fundamental Software Development: Intelligent Software Engineering* · Instructor: Liu Jiakun · HIT Global Summer School, July 2026 Repository: `code-review-ai/` · Individual lab reports: `lab1.tex/.pdf`, `lab2.tex/.pdf`, `lab3.tex/.pdf`, `lab4.tex/.pdf`

Executive summary

This project builds one system in four stages, not four unrelated assignments. Each stage’s finding directly set the question the next stage had to answer:

1. **Experiment 1** mined 1,494 real PRs across 5 repositories and established two facts the rest of the project is built on: a real, moderate class imbalance (72.0% merge rate) that makes raw accuracy a misleading metric, and a real, substantial AI-authored population (25.4% of PRs) that behaves differently enough from human-authored PRs (65.3% vs. 74.2% merge rate) to deserve its own experiment.
2. **Experiment 2** trained SVM/Random Forest merge predictors on human-written code and set a concrete performance bar under a strict, leakage-free feature contract: 76.8% accuracy, 42.5% not-merged recall (`rf_v1_balanced`). It also showed something non-obvious — a *stricter*, more leakage-resistant feature set (V2) can *beat* the literal-spec one (V1), because the room freed up by dropping two weak features gets filled by one strong one (author history).
3. **Experiment 3** asked whether a zero-training LLM could reach that same bar using nothing but a prompt. The answer is yes, but only *conditionally*: the single best LLM configuration matches the trained model’s accuracy, macro-F1, and confusion matrix **exactly** — but that peak is one specific (context, prompt) pairing out of sixteen; the pooled average across all of them falls far short (macro-F1 0.525 vs. 0.816). Prompt engineering, in other words, is not a nicety — it is most of the difference between matching a trained classifier and losing to it.
4. **Experiment 4** then asked the sharper question: does whatever worked in Experiment 3 even transfer to a harder population — code an AI agent wrote? The finding is stark: the exact three prompt strategies that occasionally caught a bad PR on human code score **exactly 0.0 not-merged recall on AI-generated code, on every single context tier, with no exception**. Only two genuinely new strategies — self-reflection and multi-turn, which change *how* the model uses information rather than *how much* it gets — ever catch a bad AI-authored PR at all. And richer context is not monotonically better: the tier that stacks *everything* regresses back to 0.0 recall, while the single well-chosen mid-tier (repository context) hits

the highest macro-F1 of all five.

5. **The system** — a FastAPI backend and VS Code extension — does not pick a winner between ML and LLM; it encodes what the four experiments actually found. A **fast** mode always runs the trained model (76.8% accuracy, cheap, free, the reliable floor). A **deep** mode is opt-in and runs Experiment 4's own best-validated configuration (**multi_turn @ diff_repo_context**) against a live, quota-limited LLM — the ceiling, used only when a user chooses to spend real API quota for it.

Every number in this document is pulled from a committed artifact (**results/tables/*.json** or a compiled lab report), not estimated — see each section's source pointers.

1. Introduction and motivation

Project II asks for one system built in five parts: a code review dataset, trained review models, merge-prediction results, generated review comments, and a working developer tool wrapping all of it. The temptation in a project shaped like this is to treat each part as an independent deliverable — mine some data, train a model, try an LLM, wrap it in a UI, done. This project was deliberately built the other way: **each stage's job was to answer a question the previous stage's results actually raised**, so that reading the four lab reports in order tells a coherent story rather than four loosely related ones. Sections 3–6 below summarize each experiment; Section 7 makes the throughline between them explicit; Section 8 covers the system that resulted.

2. System architecture

```
GitHub (5 repos)
  | GraphQL + REST mining, cached to disk
  v
[1] Raw PR dataset (1,494 PRs) --stats/plots--> Lab Report 1
    | filter: human-written (1,114) vs AI-generated (380)
    v
[2] Feature engineering (AST/CFG, diff, text)
    |
    +--> SVM / Random Forest --> merge prediction --> Lab Report 2
        | (rf_v1_balanced: 76.8% acc, 42.5% not-merged recall)
        |
[3] Context + prompt library (4 contexts x 4 strategies, human code)
    |
    +--> LLM: merge prediction + review comments --> Lab Report 3
```

```

|          (peak matches rf_v1 exactly; pooled average well below it)
|
[4] Context augmentation + advanced prompts (5 x 5, AI-gen. code)
|
+--> LLM: merge prediction + review comments --> Lab Report 4
|      (only self-reflection/multi-turn catch a bad AI PR)
v
[5] FastAPI backend (src/api/main.py)
    fast mode = [2] 's rf_v1_balanced (always run)
    deep mode = [4] 's best config: multi_turn @ diff_repo_context
    v
[6] VS Code extension (vscode-extension/)
    merge-probability gauge, click-to-jump comments, inline highlights

```

Each numbered stage is a self-contained module — the extension only ever talks to the FastAPI layer, never to the mining/ML code directly, and Experiment 2’s model doesn’t care how Experiment 3/4’s prompts are worded. That separation is what let each stage be built, tested, and reported on independently while still composing into one system.

3. Experiment 1 — the dataset

Full report: lab1.tex / lab1.pdf (15pp). Source: exp1_summary.json.

Five actively-maintained repositories were chosen deliberately, not arbitrarily: a live GitHub search confirmed each had a real, non-trivial population of PRs authored by GitHub’s own Copilot coding agent before the shortlist was finalized. Because a pure random sample of 300 closed PRs per repo would contain few or zero AI-authored PRs by chance, sampling was stratified — up to 100 agent-authored PRs per repo, filled out with recent closed PRs — a documented design decision, not a hidden shortcut.

Real numbers. 1,494 of 1,500 targeted PRs were retrieved. The 6-PR shortfall has two documented causes: 5 PRs permanently excluded after exhausting a retry budget on genuine server-side 502s, plus 1 `microsoft/vscode` slot lost to pagination drift that left that repo’s 300-entry sample list holding only 299 distinct PRs — neither silently dropped. **1,075 merged / 419 not merged — a 72.0% pooled merge rate**, ranging 63.0–81.9% by repository. **380 PRs (25.4%) are AI-authored**, ranging from 7.7% (`semantic-kernel`) to 37.5% (`aspnetcore`) by repository — reflecting how aggressively each project’s maintainers have adopted the Copilot coding agent, not an inherent property of the codebase. AI-authored PRs merge at **65.3%** vs. human-authored at **74.2%** pooled — though that direction is *not* consistent per repository (AI-authored PRs merge *more* often than human-authored ones in two of the five repos), a caveat the report states explicitly rather than let the pooled number imply a

universal effect. 71.2% of all PRs had at least one AI reviewer touch them (`copilot-pull-request-reviewer` overwhelmingly).

A three-tier AI-authorship detection system — GitHub’s own structural Bot/User typing as the primary signal, a curated allowlist to separate real AI agents from non-AI automation (`dependabot`, `renovate`, ...), and a lower-confidence secondary signal for human-opened, AI-assisted PRs — was manually spot-checked against 59 distinct flagged PRs (68 individual assessments). **Zero false positives were found.**

Why this matters downstream. Two facts from this experiment shape everything after it: the 72.0% merge rate is exactly the class imbalance Experiment 2’s evaluation design (class-weighting, minority-recall reporting) exists to handle, and the 380-PR AI-authored population is the population Experiment 4 exists to study — a population this experiment already showed merges at a measurably different rate than human-written code.

4. Experiment 2 — trained ML models (human-written code)

Full report: lab2.tex / lab2.pdf (12pp). Analysis: exp2_model_evaluation.md. Source: exp2_metrics.json.

The 1,114 human-written PRs from Experiment 1 were vectorized into three feature variants, each frozen *before* any model was trained: **V1** (the literal spec — code-structure, code-modification, textual features, explicitly excluding anything from the review process itself), **V2** (V1 minus two features that keep growing during review, plus leakage-safe author-history features), and **V0** (V1 plus the forbidden review-process signals — built solely to *measure* how much they inflate accuracy, never used as a real result). AST/CFG structural features used real tooling for Python (`ast` + `staticfg`) and a tree-sitter-based approximation for TypeScript/C#, with coverage gaps (only 71.8% of PRs contain a supported-language file) honestly zeroed rather than imputed.

Real numbers. SVM (RBF, grid-searched) and Random Forest (grid-searched) were trained on a per-repository time-based 890/224 split, plain and `class_weight='balanced'` — 12 models total. The chosen “best” model, by the experiment’s own stated criterion (minority-class recall, not raw accuracy — see Section 7), is **rf_v1_balanced: 76.8% accuracy, 42.5% not-merged recall**, a deliberate trade against `rf_v1_plain`’s higher-accuracy-lower-recall 82.1%/25.0%.

Two findings are more interesting than the headline numbers themselves:

- **The leakage demonstration is not asserted, it’s measured.** V0 (the leaky superset) reaches 97–99% ROC-AUC — a dramatic, model-

independent jump from V1’s 0.56–0.68 — and Random Forest’s own feature importances show why: `p_has_approved` alone accounts for **41.6%** of V0’s total importance. An approval essentially *is* the merge decision; Experiment 2’s ban on review-process features isn’t a rule taken on faith, it’s a rule with its exact failure mode measured and shown.

- **Stricter is not always worse.** V2 — the pre-review-strict variant — *beats* V1 on every one of 4 matched model/class-weight configurations, because the two features it drops were weak to begin with (neither is in V1’s top-3) and the author-history pair replacing them is real, non-leaky signal, making `author_prior_merge_rate` V2’s single most important feature (10.0%) — more than any individual code-derived measurement. A leakage-averse framing turned out to be an opportunity to find a better feature, not just a more honest, weaker one.

In the compliant variants, M (modification) and T (textual) features dominate roughly evenly (44.2%/40.0% in V1); S (structure) trails (15.8%) — not because structure is uninformative, but because the AST/CFG pipeline can only honestly measure it for 71.8% of PRs, and even among those, a real, root-caused `staticfg` bug (a removed `ast` attribute) degrades ~27% of otherwise-valid Python hunks to zero. This gap is stated as a traced, upstream cause, not a shrug.

5. Experiment 3 — LLM review of human-written code

Full report: lab3.tex / lab3.pdf (16pp). Analysis: exp3_model_evaluation.md. Source: exp3_metrics.json.

Four context tiers (diff-only / +description / +commit-message / +metadata) crossed with four *mechanistically distinct* prompt strategies (zero-shot, few-shot, Chain-of-Thought, role-based — not four rewordings of one instruction) over the two tasks the lab guide names (merge prediction, review-comment generation), scored on **the identical 224-PR held-out test split Experiment 2 used**, so the ML-vs-LLM comparison is genuinely apples-to-apples. The provider actually used is **Groq’s free tier** (`llama-3.1-8b-instant`); the plan’s original choice, Gemini, proved unusable without payment from this account/region even after exhausting every free workaround (documented in full in `lab3.tex`’s Problems section).

Free-tier throughput (6,000 tokens/minute, 500,000/day) makes the full 224-PR grid infeasible, so the run covers a **repo-stratified 16-PR subset**, complete: **512/512 cells, 0 failures**. Those 16 PRs are 14 merged : 2 not-merged, so raw accuracy is close to uninformative on its own (an always-MERGE model scores 87.5% for free) — the report reads minority-class recall and macro-F1

throughout, and says so explicitly rather than letting a high accuracy number stand unchallenged.

The headline finding — peak parity, not superiority. The single best LLM configuration, reached independently by two different (context, strategy) pairings (`diff_commit_message+few_shot` and `diff_metadata+role_based`), scores **93.75% accuracy, 0.816 macro-F1** — an **identical confusion matrix** (14 TP, 1 FP, 1 TN, 0 FN) to Experiment 2’s own best trained models on this same 16-PR subset. A zero-training, prompt-only approach reaches exactly the operating point a grid-searched classifier reaches. But that is the *peak*, not the *average*: pooled across all sixteen configurations, the LLM’s macro-F1 is **0.525** — well below 0.816 — because eleven of the sixteen configurations never predict CLOSE at all. The competitiveness the headline number shows is **entirely contingent on choosing the right prompt and context**, which is the central, load-bearing finding this experiment produces for Section 7 below.

Secondary findings, each concrete rather than generic: `few_shot` is the *worst* strategy on raw accuracy (0.750) but the *best* on minority recall (0.250 vs. 0.000 for zero-shot/CoT) — the class-imbalance accuracy-vs-recall trade-off from Experiment 2 reappearing as a prompt-design lever rather than a `class_weight` one. `role_based` is the best merge-prediction strategy on average (0.891 accuracy) but ranks *last* on review-comment quality (ROUGE-L), direct evidence the best prompt is task-dependent, not universal. BLEU/ROUGE against real human comments are (expectedly) very low in absolute terms — the report pairs the numeric score with a qualitative side-by-side, since an LLM’s review can be accurate and on-topic while sharing almost no n-grams with a terse two-word human comment.

6. Experiment 4 — improving review of AI-generated code

Full report: lab4.tex / lab4.pdf (14pp). Analysis: exp4_model_evaluation.md. Source: exp4_metrics.json.

Same machinery as Experiment 3, retargeted at the **380 AI-authored PRs** from Experiment 1, with two axes deliberately widened: **five** context tiers (adding repository-level context and linked-issue text, and stacking everything into a **complete** tier) instead of four, and **five** prompt strategies — the three that generalize unchanged from Experiment 3 (role-based, few-shot, CoT), plus two genuinely new reasoning mechanisms Experiment 3 never used: **self-reflection** (draft-then-critique-then-finalize) and **multi-turn** (reveal context progressively, reconsider after). Both render to a real multi-turn conversation rather than a single-shot prompt, because their mechanism *is* the turn sequence — a single prompt cannot express “critique the draft you just wrote.”

The same shared, account-wide Groq quota that Experiment 3 also draws from meant this run reached **3 fully-complete PRs** (2 merged, 1 not-merged) plus a 4th partial, **148/384 cells, 0 failures** — a small, honestly-stated sample, read as directional and paired (the *same* few PRs across every tier/strategy) rather than a converged estimate.

The headline finding — prompting that works on human code stops working on AI code. On the three strategies shared with Experiment 3, results are unambiguous: `role_based/few_shot/cot` score **exactly 0.0 not-merged recall on every single context tier, with no exception** — reviewing AI-generated code with the very prompts that occasionally worked on human code (Section 5: `few_shot` averaged 0.250 recall there). **Only the two new strategies ever catch a bad AI-authored PR at all: `self_reflection`** reaches the highest recall (0.60) but pays for it in accuracy (0.375), because its critique turn is skeptical of the draft *regardless of whether the draft was actually right* — a real transcript in the report shows the critique talking a correct MERGE call into an incorrect CLOSE one, included precisely because it is honest evidence of the mechanism, not a flattering cherry-pick. `multi_turn` is the best-balanced strategy of all five (macro-F1 0.486, recall 0.50, without self-reflection’s accuracy collapse), because it only revises its diff-only judgment when the newly-revealed context actually contradicts it, rather than second-guessing indiscriminately.

The second finding — richer context is not monotonically better. Adding repository-level context (a no-fetch approximation: co-changed files, enclosing function/class names from git hunk headings) is what moves the needle: not-merged recall goes from **0.0** (bare diff) to **0.4**, and macro-F1 peaks there (0.498) among all five tiers. But the **complete** tier — which stacks *every* context source at once — regresses back to **0.0** recall. Piling on more information did not help this model-and-sample; the report names this as a genuinely open question for a larger model and sample, not a settled conclusion.

The two axes are separable by design, and the data confirms it: the *same* repository-context tier drives self-reflection/multi-turn to their best recall while leaving role-based/few-shot/CoT at 0.0 on that identical tier. Context augmentation fixes an information deficit; prompt optimization fixes a disposition deficit (the model defaulting to “looks locally fine, so merge” regardless of what it’s shown). Neither one alone was sufficient on this sample — both were necessary.

7. Cross-experiment synthesis — the throughline

This is the narrative Section 8.2 of the project plan asks for explicitly: not four summaries, but what each stage’s result actually did to the next stage’s question.

	Experiment 2 (ML)	Experiment 3 (LLM, human code)	Experiment 4 (LLM, AI code)
Best config	<code>rf_v1_balanced</code>	<code>few_shot / role_based</code> (2 tied peaks)	<code>multi_turn</code>
Accuracy	76.8%	93.75% (peak) / 84.8% (pooled)	50.0%
Not-merged recall	42.5%	50.0% (peak) / 9.4% (pooled)	50.0%
Macro-F1	—	0.816 (peak) / 0.525 (pooled)	0.486 (best strategy)
Training cost	Grid search on 890 PRs	None (prompt only)	None (prompt only)
Inference cost	Milliseconds, free	~60–1,090 ms/call, real API quota	~120–1,240 ms/call, 1–2 calls, real API quota

(Exp3/Exp4 accuracy/recall are not directly comparable to each other or to Exp2 as raw numbers — the populations, samples, and base rates all differ; the comparable quantity across all three is the relative pattern each reveals, which is what the rows below draw out. Both inference-cost ranges are stated on one basis — the mean server-side latency of each (task $\times$ strategy) group — so the two columns are like-for-like; Experiment 4’s wider top end is the cost of *self_reflection/multi_turn* spending two API calls per review. “Pooled” means one confusion matrix over all 16 configs’ predictions, not the mean of the 16 per-config scores: for accuracy and recall the two coincide exactly, since every config sees the same 16 PRs, but macro-F1 is not linear in the confusion counts, so the mean-of-configs macro-F1 is 0.509 against the pooled 0.525 — the same finding either way.)

The throughline, stage by stage:

Experiment 1 didn’t just produce a dataset — it produced two design constraints every later stage had to answer: a class imbalance robust enough that “just report accuracy” would be actively misleading, and an AI-authored population large enough (380 PRs, 25.4% of the total) to be worth a whole separate experiment rather than a footnote.

Experiment 2 answered the class-imbalance constraint directly (class-weighting, minority-recall reporting, a time-based split) and, in doing so, produced a concrete number — 76.8% accuracy / 42.5% recall — that is not just a result but a **target**: the bar any zero-training alternative would need to clear to be worth considering.

Experiment 3 cleared that bar — at its peak. But the *manner* in which it cleared

it is the real finding: not by being reliably better, but by being *exactly as good, once, when the prompt and context were chosen correctly*, while collapsing to well below the bar on most other configurations. That result reframes the entire question Experiment 4 has to ask: not “can an LLM review code,” which Experiment 3 already answered, but “does whatever made Experiment 3’s prompts occasionally work survive contact with a harder, different population?”

Experiment 4’s answer is no, cleanly and completely for the shared strategies (0.0 recall, zero exceptions), which is itself evidence about *why* Experiment 3’s peak was narrow: the strategies that worked on human code were exploiting patterns specific to human-authored PRs (author’s own stated intent in a commit message, a maintainer persona’s risk calibration tuned to human contribution norms) that do not transfer to AI-authored ones. What *does* transfer is a change in *mechanism*, not *content* — self-reflection and multi-turn do not know anything zero-shot doesn’t; they use what they know differently, by forcing a second, skeptical pass. That is the single most exportable finding across all four experiments: **for AI-generated code, changing how a model reasons matters more than changing what it’s shown** — repository context helps, but only once paired with a strategy disposed to actually reconsider in light of it.

8. The system — backend and VS Code extension

Full write-ups: `api_design.md`, `vscode_extension_design.md`, `portfolio_readiness.md`.

The system does not pick a winner between the trained model and the LLM — it encodes what Sections 4–6 actually found about each one’s reliability.

Fast mode (`src/api/main.py`, default) always runs `rf_v1_balanced` — chosen not for raw accuracy but for the same reason Experiment 2 itself argues for: it is the compliant, non-leaky, minority-recall-aware model, and it is free and instant. A purpose-built feature-extraction path (`src/api/ml_features.py`) re-derives Experiment 2’s exact V1 feature vector from a raw diff that has no `pr_id` — reusing the AST/CFG pipeline unchanged — so the backend works on any diff, not just PRs already in the mined dataset.

Deep mode is opt-in, because it spends real, shared, rate-limited LLM quota — the exact resource Experiment 3/4’s own honestly-small samples are a direct consequence of. It runs Experiment 4’s own best-*balanced* configuration, `multi_turn @ diff_repo_context`, not just because that’s the configuration with the best macro-F1, but because Section 6’s separable-axes finding justifies it structurally: repository context (built with zero extra network calls, from the diff’s own git hunk headings, the same no-fetch approximation Experiment 4 used) is the context tier proven to help, and multi-turn is the strategy proven to use it without self-reflection’s accuracy cost. Any LLM failure — quota ex-

haustion, a network error — degrades the response to the fast-mode result plus a warning field, never a 500; the design decision explicitly rejected a canned fallback response as worse than an honest “unavailable.”

The VS Code extension (`vscode-extension/`) surfaces both: a merge-probability gauge, and — since the backend returns review comments as prose with no location, exactly the output shape Experiment 3/4’s prompt contracts produce — a client-side heuristic (`anchoring.ts`) that resolves each comment to a `file:line` by matching it against the diff, with its own confidence level surfaced in the UI rather than hidden. Real deep-mode LLM output was fed through this heuristic during development, and it changed the design: the first pass resolved 3 of 4 real comments (missing ones that named a function defined on an unchanged *context* line, not an added one); indexing context lines as a secondary source raised that to 4 of 4.

Verification, at every layer. 414 Python tests cover mining, feature extraction, ML training/evaluation, and the LLM pipeline (LLM-provider tests mock the HTTP layer entirely — no test depends on live API access). The extension has 49 unit tests for its `vscode`-independent logic and 7 integration tests that run inside a real, unmodified VS Code instance — a real `git diff`, a real backend call, a real results panel, and a real click-to-jump navigation, not mocks. The packaged `.vsix` was independently verified by installing it into an isolated VS Code profile via the CLI and diffing its extracted code against the exact code the integration suite had just run against: byte-identical.

9. Limitations and threats to validity

Stated together here because they recur across experiments and interact:

- **Experiment 3/4’s LLM samples are small, by a real external constraint, not a design choice.** Groq’s free tier caps at 500,000 tokens/day, shared across the whole account — Experiment 3 reached 16 of 224 possible test PRs; Experiment 4 reached 3 fully-complete AI-authored PRs. Every LLM-side number in Sections 5–6 is a directional, paired observation on a stated sample, not a converged estimate, and both lab reports say so explicitly rather than presenting a small sample as more than it is.
- **The `complete_se_context` regression (Section 6) and the exact peak-parity pairing (Section 5) both need a larger sample to confirm as general rather than sample-specific.** Two negatives (Exp3) and one negative (Exp4, at the fully-complete-PR level) are enough to separate configurations from each other but not enough to rule out sampling noise on any single comparison.
- **`--fetch-issues` was not enabled in the Experiment 4 grid run** (an operational oversight caught after quota was exhausted), so the

`diff_issue/complete` tiers’ real effect — as opposed to their “no issue data” baseline — is still genuinely unmeasured.

- **AST/CFG structural features have honestly-documented, root-caused coverage gaps** (71.8% language coverage; a real `staticfg` library bug degrades ~27% of otherwise-valid Python hunks) rather than silently-imputed values — but the gap itself means the S feature category’s true predictive ceiling, if fully measurable, is unknown.
- **The extension’s comment-to-location anchoring is a text heuristic**, not a guarantee: it resolves what the model *wrote* against the diff, which is not always what the model *meant*. Its precision levels are surfaced in the UI specifically so this limitation is visible to a user, not hidden behind a confident-looking green checkmark.
- **No git remote is configured for this repository in the environment this project was built in.** The CI workflow (`.github/workflows/ci.yml`) has been verified by running its exact commands locally, but never on a real hosted GitHub Actions run.

None of these limitations were discovered late — each is named in the relevant lab report’s own Problems/Reflections sections at the point the experiment that surfaced it was run, and each report proposes the concrete next step that would resolve it (a larger sample, `--fetch-issues` enabled, a stronger base model, real repository-fetch instead of the git-heading approximation).

10. Conclusion

Four experiments, read in order, tell one story: a real dataset revealed a real class imbalance and a real AI-authored population worth studying separately; a trained model set a concrete, honestly-evaluated bar under a strict feature contract; an LLM matched that bar exactly, but only when prompt and context were chosen well, revealing that matching a trained classifier with zero training is possible but fragile; and the same prompting that achieved that fragile match on human code failed *completely* on AI-generated code, until the model was asked to reason differently — not just given more to reason about. The system built from these findings does not resolve the tension between the reliable trained model and the occasionally-superior-but-volatile LLM by picking one; it offers both, with the LLM path specifically configured to be the one finding across four experiments’ worth of evidence actually supports for its target population. That is what makes this a system built in stages rather than four assignments turned in together — each stage changed what the next one had to build.